

1. For every system call, changes have been made to the following files.

- “kernel/syscall.h”
- “kernel/syscall.c”
- “kernel/usys.S”
- “user/user.h”
- “kernel/sysfile.c” / “kernel/sysproc.c”

All the helper functions have been included in “proc.c” to implement the system call “getsyscount” a separate file “user/syscount.c” has been included.

```
int get_syscall_count(int mask)
{
    struct proc *p = myproc();
    int syscall_num = 0;

    while (mask > 1)
    {
        mask >>= 1;
        syscall_num++;
    }

    return p->syscall_count[syscall_num];
}
```

Additional System Call Specifications

1. Signal Alarm (sys_sigalarm)

- **Feature Description:** The sys_sigalarm() system call allows a process to set up a periodic alarm. This alarm triggers after a specified interval of ticks and invokes a user-defined signal handler.
- **Key Components:**
 - **Parameters:**
 - interval: The number of ticks after which the alarm will trigger.

- handler: A function pointer to the user-defined signal handler.
- **Validity Check:**
 - The `is_valid_user_addr(uint64 addr)` function verifies that the handler function pointer is a valid user-space address.
 - If the address is invalid (out of range or points to a non-mapped region), the call returns -1 (error).
- **Behavior:**
 - If valid, the system sets up the alarm with the specified interval and assigns the handler function. The alarm ticks are counted using `p->tick_counter`, and when the interval is reached, the signal handler is triggered.

2. Signal Return (`sys_sigreturn`)

- **Feature Description:** The `sys_sigreturn()` system call allows a process to return to its normal execution state after handling a signal alarm.
- **Key Components:**
 - **Restoration of Process State:**
 - When a signal alarm is triggered, the original process state is saved in `p->saved_tf`. This system call restores the saved state by copying it back into the process's trapframe (`p->trapframe`).
 - **Memory Cleanup:**
 - After restoring the state, the saved trapframe is freed (`kfree(p->saved_tf)`), and the alarm handle flag (`p->alarm_handle`) is reset to prevent further alarms until a new one is set.
 - **Return Value:**
 - It returns the value stored in `p->trapframe->a0`, which represents the value returned by the signal handler.

3. Lottery-Based Scheduling - `settickets()` System Call

- **Feature Description:** The `sys_settickets()` system call is used to assign a specific number of tickets to a process for lottery-based scheduling (LBS).
- **Key Components:**
 - **Parameter:**
 - `n`: The number of tickets to assign to the calling process.
 - **Validation:**
 - If the number of tickets (`n`) is less than 1, the system call returns -1 (error) since a process must have at least one ticket.
 - **Ticket Assignment:**

- If valid, the process is assigned n tickets via p->tickets. The more tickets a process has, the higher its chances of being scheduled in the LBS algorithm.
- **Note:**
 - The ticket assignment only occurs if the Lottery-Based Scheduling (LBS) is enabled (`#ifdef LBS`).

4. System Call Count (`sys_getsyscount`)

- **Feature Description:** The `sys_getsyscount()` system call allows a process to retrieve the count of a specific system call based on a given mask.
- **Key Components:**
 - **Parameter:**
 - The mask value is extracted from the process's trapframe (p->trapframe->a0), which indicates which system call count to retrieve.
 - **Counting Logic:**
 - The `get_syscall_count(int mask)` function is used to determine which system call is being requested and returns the number of times that system call has been invoked by the process.
 - **Use Case:**
 - This feature is particularly useful for monitoring and debugging processes, as it allows fine-grained tracking of specific system call usage.

Ques. What is the implication of adding the arrival time in the lottery based scheduling policy? Are there any pitfalls to watch out for? What happens if all processes have the same number of tickets?

Ans. **Implication of Adding Arrival Time:**

- **Fairness:** Helps older processes get a better chance of scheduling, preventing starvation.
- **Pitfalls:** Could overly favor older processes, making scheduling less random.
- **Complexity:** Adds overhead by considering both arrival time and ticket count.

If All Processes Have the Same Number of Tickets:

- **Equal Probability:** All processes get the same chance to run, making scheduling random.
- **No Prioritization:** No process has priority, so long-waiting processes might suffer.

Pitfalls:

- **Starvation:** Risk if arrival time isn't considered.

- **Overhead:** Increases computational cost by tracking arrival times.

Solutions:

- **Age Factor:** Use waiting time to avoid starvation.
- **Hybrid Scheme:** Combine ticket count and other factors for fairness.

Ques :- Include the performance comparison between the default and 2 implemented scheduling policies by showing the average waiting and running times for processes. Set the processes to run on only 1 CPU for this purpose. Use the `schedulertest` command to get this information.

Ans :- ROUND-ROBIN = Average rtime 36, wtime 195

LBS = Average rtime 30, wtime 155

MLFQ = Average rtime 25, wtime 143

1. Multi-Level Feedback Queue (MLFQ) Scheduling

- **Feature Description:** You have added MLFQ scheduling functionality to prioritize processes based on their execution behavior. This system dynamically adjusts the priority of processes based on the time slices they use.
- **Key Methods:**
 - `push_to_queue(struct proc *p, int priority)`: Adds a process to the appropriate priority queue based on its priority value. The time slice is determined by the priority level.
 - `pop_from_queue(int priority)`: Removes and returns a process from a priority queue.
 - `boost_priority(void)`: This function boosts all runnable processes to the highest priority after a certain number of ticks (96 in your case) to prevent starvation.
- **Scheduling Logic:**
 - The `mlfq_scheduler()` is the main function for scheduling processes based on their priority. If a process exhausts its time slice, it is moved to a lower priority queue. If the total ticks exceed a threshold (e.g., 96), the priorities of all processes are reset to the highest level.

2. Lottery-Based Scheduling (LBS)

- **Feature Description:** You've implemented a lottery-based scheduling algorithm, an alternative to MLFQ. In LBS, each process is assigned a number of tickets, and a random ticket is drawn to select which process runs.
- **Key Methods:**
 - `lbs_scheduler(void)`: This is the scheduler implementation for LBS. It calculates the total number of tickets for runnable processes, picks a random number, and runs the process corresponding to the winning ticket.

- random_at_most(int max): Generates a random number between 1 and max to simulate the lottery draw.

